

■ Conversational RAG with Memory Architecture

Complete Technical Blueprint, Architecture Diagram & Python Implementation Guide for Smart Expense Tracker

Executive Summary: This blueprint details how to build a 100% Free, Local **Retrieval-Augmented Generation (RAG) system with Short-Term Conversational Memory** for financial expense tracking. It combines *Local Vector Similarity Search* for semantic transactions with *SQL Aggregation* for exact mathematical precision and *Session Memory* for seamless multi-turn conversations.

1. System Architecture Blueprint

Step / Layer	Component & Responsibility	Tech Used
1. Input & Session	User query via Voice Mic / Chat with active session ID.	Web Speech / JS
2. Memory Buffer	Tracks last 6-8 conversation turns to resolve pronouns ('it', 'those').	Django Session / Dict
3. Query Rewriter	Reformulates follow-up questions into standalone search prompts.	Rule / Gemini Flash
4. Hybrid Retriever	Semantic Vector Search (Top-K) + SQL Aggregation (Sum/Count/Avg).	TF-IDF / Dense + Postgres
5. Context Synthesizer	Grounded prompt formulation with strict financial context.	Python Engine
6. Dual Output	Generates grounded markdown chat response + streams neural speech.	Gemini + Edge-TTS

2. Conversational Memory Architecture (Short-Term Buffer)

Standard RAG handles each query in isolation. The Memory Buffer maintains a rolling window of recent dialog turns, allowing the user to ask follow-up questions naturally (e.g. 'What about last month?', 'Which of those was highest?').

```

# File: ai_engine/memory.py
from dataclasses import dataclass, field
from typing import List, Dict
import time

@dataclass
class ConversationTurn:
    role: str # 'user' or 'assistant'
    content: str
    timestamp: float = field(default_factory=time.time)

class SessionMemoryBuffer:
    def __init__(self, max_turns: int = 8):
        self.max_turns = max_turns
        self._history: List[ConversationTurn] = []

    def add_user_message(self, text: str):
        self._history.append(ConversationTurn(role='user', content=text))
        self._trim()

    def add_bot_message(self, text: str):
        self._history.append(ConversationTurn(role='assistant', content=text))
        self._trim()

    def get_context_tuples(self) -> List[Dict[str, str]]:
        return [{'role': t.role, 'text': t.content} for t in self._history]

    def _trim(self):
        if len(self._history) > self.max_turns * 2:
            self._history = self._history[-(self.max_turns * 2):]

    def rewrite_query_with_context(self, current_query: str) -> str:
        """Condenses follow-up queries using prior turn context."""
        if not self._history:
            return current_query

        last_turns = self._history[-4:]
        pronouns = ['it', 'those', 'them', 'that', 'same', 'there', 'what about', 'how about']
        is_followup = any(p in current_query.lower() for p in pronouns) or len(current_query.split()) < 4

        if is_followup and len(last_turns) >= 2:
            prev_user = [t.content for t in last_turns if t.role == 'user'][-1]
            return f"{prev_user} -> follow up: {current_query}"
        return current_query

```

3. Local Vector Index & Semantic Search (Zero Cost)

To achieve zero cost and instant search, we build a local vector index using high-performance TF-IDF sublinear vectors or dense embeddings with Cosine Similarity. When bank statements are uploaded, every transaction chunk is indexed instantly.

```
# File: ai_engine/rag.py
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from typing import List, Dict, Any

class LocalFinancialVectorStore:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(
            ngram_range=(1, 3),
            sublinear_tf=True,
            token_pattern=r'(?u)\b\w+\b'
        )
        self.documents: List[str] = []
        self.metadata: List[Dict[str, Any]] = []
        self.matrix = None

    def build_index(self, transactions: List[Dict[str, Any]]):
        """Indexes a list of transactions into memory."""
        self.documents = []
        self.metadata = []
        for txn in transactions:
            # Create rich semantic chunk
            chunk = (
                f>Date: {txn.get('txn_date')} | "
                f>Payee/Narration: {txn.get('description')} | "
                f>Amount: Rs. {txn.get('amount')} | "
                f>Type: {txn.get('txn_type')} | "
                f>Category: {txn.get('category', 'Uncategorized'))}"
            )
            self.documents.append(chunk)
            self.metadata.append(txn)

        if self.documents:
            self.matrix = self.vectorizer.fit_transform(self.documents)

    def search(self, query: str, top_k: int = 5) -> List[Dict[str, Any]]:
        """Performs cosine similarity search against indexed transactions."""
        if self.matrix is None or not self.documents:
            return []

        query_vec = self.vectorizer.transform([query])
        scores = cosine_similarity(query_vec, self.matrix).flatten()

        top_indices = np.argsort(scores)[::-1][:top_k]
        results = []
        for idx in top_indices:
            if scores[idx] > 0.05: # Relevance threshold
                results.append({
                    'chunk': self.documents[idx],
                    'score': float(scores[idx]),
                    'txn': self.metadata[idx]
                })
        return results
```

4. Hybrid Query Router (Vector Search + SQL Math)

Financial systems must never guess mathematical sums. The Hybrid Router sends numerical and aggregate questions to PostgreSQL/Django ORM (`Sum`, `Avg`, `Count`) while sending vendor and semantic discovery questions to the Vector Store.

```
# File: ai_engine/hybrid_router.py
import re
from django.db.models import Sum, Count, Avg
from tracker.models import Transaction

def route_and_retrieve(user, query: str, vector_store: LocalFinancialVectorStore) -> Dict[str, Any]:
    q_lower = query.lower()
    math_triggers = ['total', 'how much', 'spent on', 'sum', 'average', 'highest', 'largest']

    # 1. Check if query asks for aggregations (SQL Engine)
    is_math_query = any(k in q_lower for k in math_triggers)
    math_summary = {}

    if is_math_query:
        qs = Transaction.objects.filter(account__user=user)
        # Check category matches
        for cat in ['Food & Dining', 'Travel & Fuel', 'Shopping', 'Bills & Utilities', 'Groceries']:
            if cat.lower() in q_lower or cat.split()[0].lower() in q_lower:
                qs = qs.filter(category__iexact=cat)

        agg = qs.aggregate(total_spent=Sum('debit'), count=Count('id'), avg_spent=Avg('debit'))
        math_summary = {
            'total_debit': float(agg['total_spent'] or 0.0),
            'transaction_count': agg['count'] or 0,
            'average_debit': float(agg['avg_spent'] or 0.0)
        }

    # 2. Perform Semantic Vector Search (Top-5 Chunks)
    vector_results = vector_store.search(query, top_k=5)

    return {
        'math_summary': math_summary,
        'semantic_chunks': [r['chunk'] for r in vector_results],
        'raw_transactions': [r['txn'] for r in vector_results]
    }
```

5. Grounded Prompt Formulation & Voice Synthesis

We inject the conversation history, mathematical calculations, and retrieved transaction chunks directly into the prompt. The response is rendered in the chat and spoken via the SET Neural Voice module.

```
# File: ai_engine/chat.py
import google.genai as genai
from config import settings

def generate_rag_response(user, user_query: str, session_memory, vector_store) -> str:
    # 1. Resolve query with conversation memory
    search_query = session_memory.rewrite_query_with_context(user_query)

    # 2. Retrieve Hybrid Context
    context_data = route_and_retrieve(user, search_query, vector_store)

    # 3. Construct Grounded Prompt
    prompt = f"""You are SET (Smart Expense Tracker AI), an expert financial assistant.
    User Name: {user.first_name or user.username}

    RELEVANT RETRIEVED TRANSACTIONS:
    {chr(10).join(context_data['semantic_chunks']) if context_data['semantic_chunks'] else 'No specific semantic matches found.'

    CALCULATED FINANCIAL DATA (SQL GROUND TRUTH):
    {context_data['math_summary']}

    RECENT DIALOG HISTORY:
    {session_memory.get_context_tuples()}

    USER QUESTION: {user_query}

    GUIDELINES:
    1. Address the user naturally by name ({user.first_name or user.username}). Never use 'boss'.
    2. Use Rupees (₹) for all currency figures.
    3. Be precise with dates, vendors, and amounts.
    4. Keep the explanation crisp, helpful, and concise.
    """

    # 4. Call Gemini (Free Tier)
    if settings.GEMINI_API_KEY:
        client = genai.Client(api_key=settings.GEMINI_API_KEY)
        response = client.models.generate_content(
            model='gemini-1.5-flash',
            contents=prompt
        )
        reply = response.text
    else:
        # Intelligent Offline Local Fallback
        reply = local_financial_fallback(user, user_query, context_data)

    # 5. Store turn in memory buffer
    session_memory.add_user_message(user_query)
    session_memory.add_bot_message(reply)

    return reply
```

6. Step-by-Step Implementation & Viva Checklist

- **Step 1:** Create `ai_engine/memory.py` with `SessionMemoryBuffer` to retain rolling multi-turn context.
- **Step 2:** Create `ai_engine/rag.py` with `LocalFinancialVectorStore` for zero-cost semantic search.
- **Step 3:** Update `tracker/views.py` upload endpoint to auto-index newly ingested transactions into the vector store.
- **Step 4:** Integrate `route_and_retrieve` in `ai_engine/chat.py` to fuse SQL math ground-truth with semantic chunks.

- **Step 5:** Stream voice output using /api/tts/ (SET Neural Voice with Pause/Continue/Stop controller).
- **Step 6:** Test multi-turn follow-ups: *'How much spent on Swiggy?'* → *'What about Zomato?'* → *'Which was largest?'*